



Data and Artificial Intelligence

Cyber Shujaa Program

Week 8 Project Report Supervised Learning: Classification Models

Student Name: Amidu Dabor

Student ID: CS-DA01-25031

Date: 7 July, 2025

TABLE OF CONTENTS

1.0	INTRODUCTION	1
2.0	TASK COMPLETION	1
2.1	LOADING AND EXPLORING THE DATASET	1
2.2	DATA PREPARATION	4
2.3	MODEL IMPLEMENTATION AND EVALUATIONS	5
2.4	MODEL COMPARISON AND SUMMARY	17
2.5	PUBLISHING AND SHARING	17
3.0	PORTFOLIO WEBSITE	18
4.0	CONCLUSION	18
5.0	REFERENCES	19

1.0 INTRODUCTION

This project demonstrates the application of supervised machine learning classification models using the Wine dataset from Scikit-learn. The project is implemented using Python and Object-Oriented Programming (OOP) principles for modular, reusable, and structured code.

The primary objective was to:

- Build and evaluate six classification models
- Perform Exploratory Data Analysis (EDA)
- Visualize decision boundaries and confusion matrices
- Compare model performances and select the best one

The models implemented include:

- Logistic Regression
- Decision Tree
- Random Forest
- K-Nearest Neighbors
- Naive Bayes
- Support Vector Machine

2.0 TASK COMPLETION

2.1 Loading and Exploring the Dataset

- The dataset was loaded using `load_wine()` from Scikit-learn.
- Exploratory steps included:
 - Viewing the first 5 rows
 - Checking for missing values
 - Viewing feature summary statistics
 - Visualizing class distribution

```
... ✓ Dataset loaded successfully.

✂ First 5 rows of the dataset:
  alcohol  malic_acid  ash  alcalinity_of_ash  magnesium  total_phenols  \
0    14.23      1.71  2.43             15.6      127.0         2.80
1    13.20      1.78  2.14             11.2     100.0         2.65
2    13.16      2.36  2.67             18.6     101.0         2.80
3    14.37      1.95  2.50             16.8     113.0         3.85
4    13.24      2.59  2.87             21.0     118.0         2.80

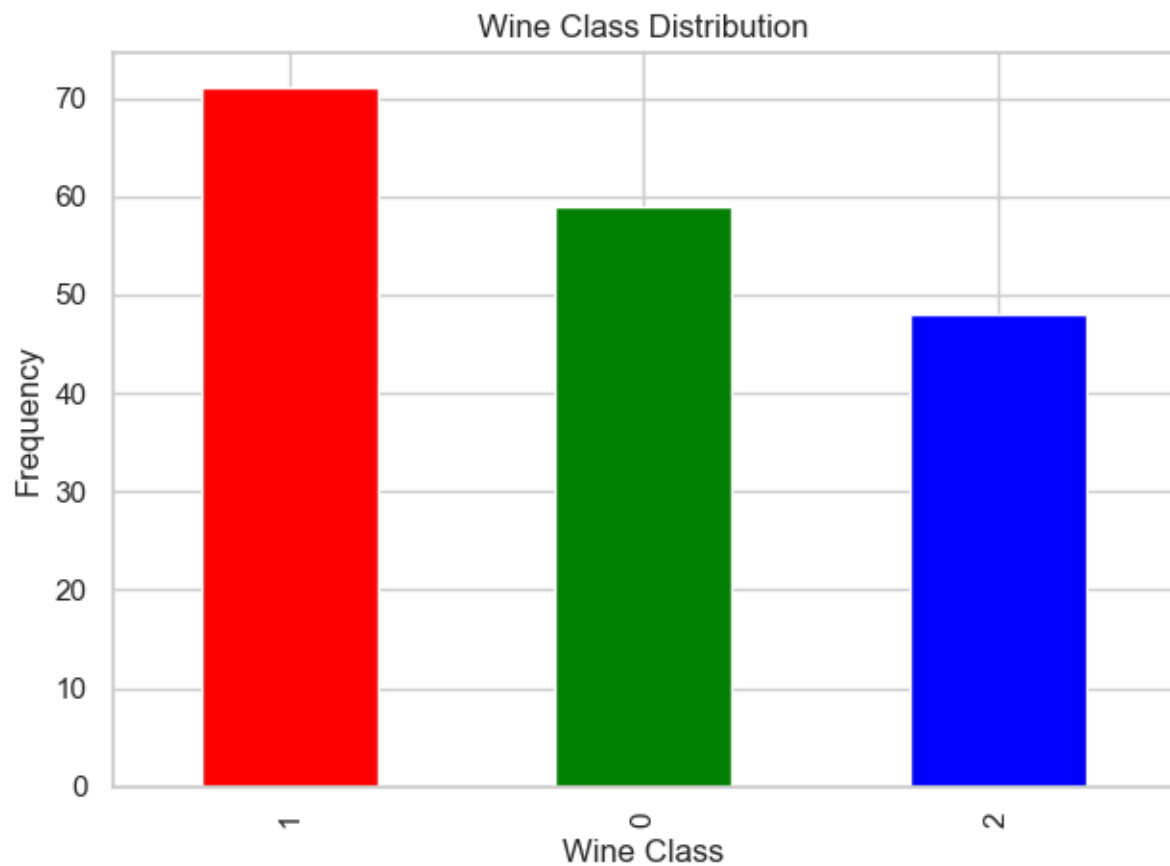
  flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity  hue  \
0         3.06                0.28             2.29             5.64  1.04
1         2.76                0.26             1.28             4.38  1.05
2         3.24                0.30             2.81             5.68  1.03
3         3.49                0.24             2.18             7.80  0.86
4         2.69                0.39             1.82             4.32  1.04

  od280/od315_of_diluted_wines  proline  target
0                        3.92    1065.0      0
1                        3.40    1050.0      0
2                        3.17    1185.0      0
3                        3.45    1480.0      0
4                        2.93     735.0      0
```

```
... 🧠 Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 14 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   alcohol                                   178 non-null    float64
1   malic_acid                               178 non-null    float64
2   ash                                       178 non-null    float64
3   alcalinity_of_ash                        178 non-null    float64
4   magnesium                                 178 non-null    float64
5   total_phenols                            178 non-null    float64
6   flavanoids                               178 non-null    float64
7   nonflavanoid_phenols                     178 non-null    float64
8   proanthocyanins                          178 non-null    float64
9   color_intensity                          178 non-null    float64
10  hue                                       178 non-null    float64
11  od280/od315_of_diluted_wines             178 non-null    float64
12  proline                                   178 non-null    float64
13  target                                    178 non-null    int64
dtypes: float64(13), int64(1)
memory usage: 19.6 KB
None
```

Summary Statistics:					
	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium \
count	178.000000	178.000000	178.000000	178.000000	178.000000
mean	13.000618	2.336348	2.366517	19.494944	99.741573
std	0.811827	1.117146	0.274344	3.339564	14.282484
min	11.030000	0.740000	1.360000	10.600000	70.000000
25%	12.362500	1.602500	2.210000	17.200000	88.000000
50%	13.050000	1.865000	2.360000	19.500000	98.000000
75%	13.677500	3.082500	2.557500	21.500000	107.000000
max	14.830000	5.800000	3.230000	30.000000	162.000000
	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins \	
count	178.000000	178.000000	178.000000	178.000000	
mean	2.295112	2.029270	0.361854	1.590899	
std	0.625851	0.998859	0.124453	0.572359	
min	0.980000	0.340000	0.130000	0.410000	
25%	1.742500	1.205000	0.270000	1.250000	
50%	2.355000	2.135000	0.340000	1.555000	
75%	2.800000	2.875000	0.437500	1.950000	
max	3.880000	5.080000	0.660000	3.580000	
	color_intensity	hue	od280/od315_of_diluted_wines	proline \	
count	178.000000	178.000000	178.000000	178.000000	
mean	5.058090	0.957449	2.611685	746.893258	
std	2.318286	0.228572	0.709990	314.907474	
min	1.280000	0.480000	1.270000	278.000000	
25%	3.220000	0.782500	1.937500	500.500000	
50%	4.690000	0.965000	2.780000	673.500000	
75%	6.200000	1.120000	3.170000	985.000000	
max	13.000000	1.710000	4.000000	1680.000000	
target					
count	178.000000				
mean	0.938202				
std	0.775035				
min	0.000000				
25%	0.000000				
50%	1.000000				
75%	2.000000				
max	2.000000				
Missing Values Check:					
alcohol	0				
malic_acid	0				
ash	0				
alcalinity_of_ash	0				
magnesium	0				
total_phenols	0				
flavanoids	0				
nonflavanoid_phenols	0				
proanthocyanins	0				
color_intensity	0				
hue	0				
od280/od315_of_diluted_wines	0				
proline	0				
target	0				
dtype: int64					

```
Class Distribution:
target
1    71
0    59
2    48
Name: count, dtype: int64
```



2.2 Data Preparation

- The first two features were selected for easier 2D visualization.
- The data was scaled using StandardScaler.
- The dataset was split into 80% training and 20% test sets.

```
45 # Standardization
46 self.sc = StandardScaler()
47 X_scaled = self.sc.fit_transform(X)
48
49 # Train/test split
50 self.X_train, self.X_test, self.y_train, self.y_test = train_test_split(
51     X_scaled, y, test_size=0.2, random_state=0
52 )
53 print("\n👉 Data has been scaled and split into training and test sets.")
54
```

...

📦 Data has been scaled and split into training and test sets.

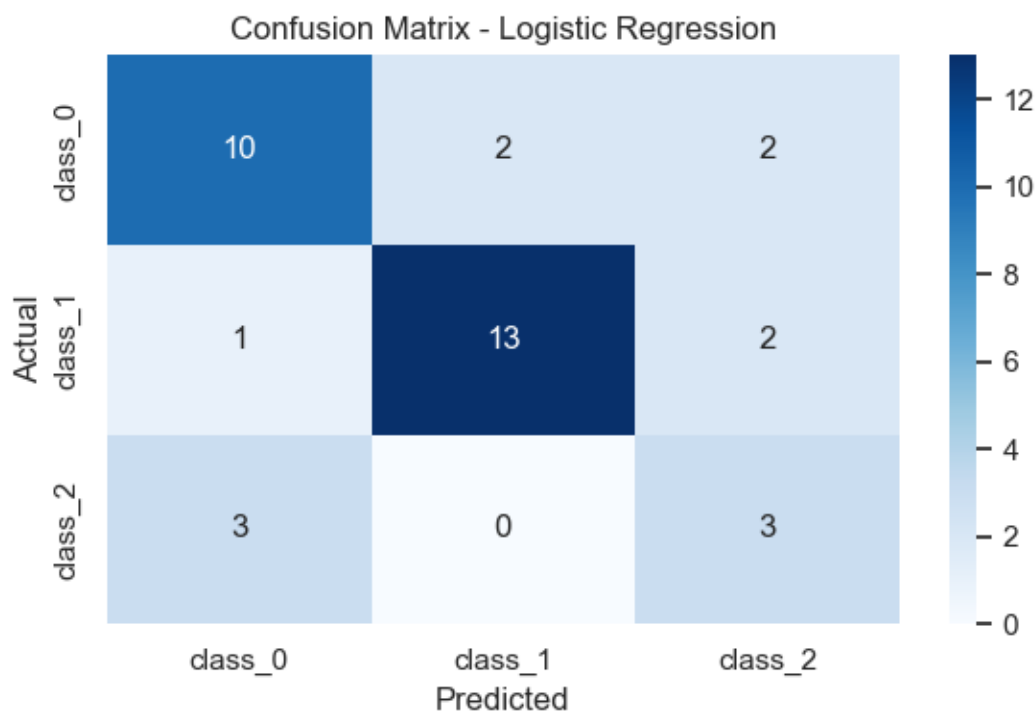
2.3 Model Implementation and Evaluations

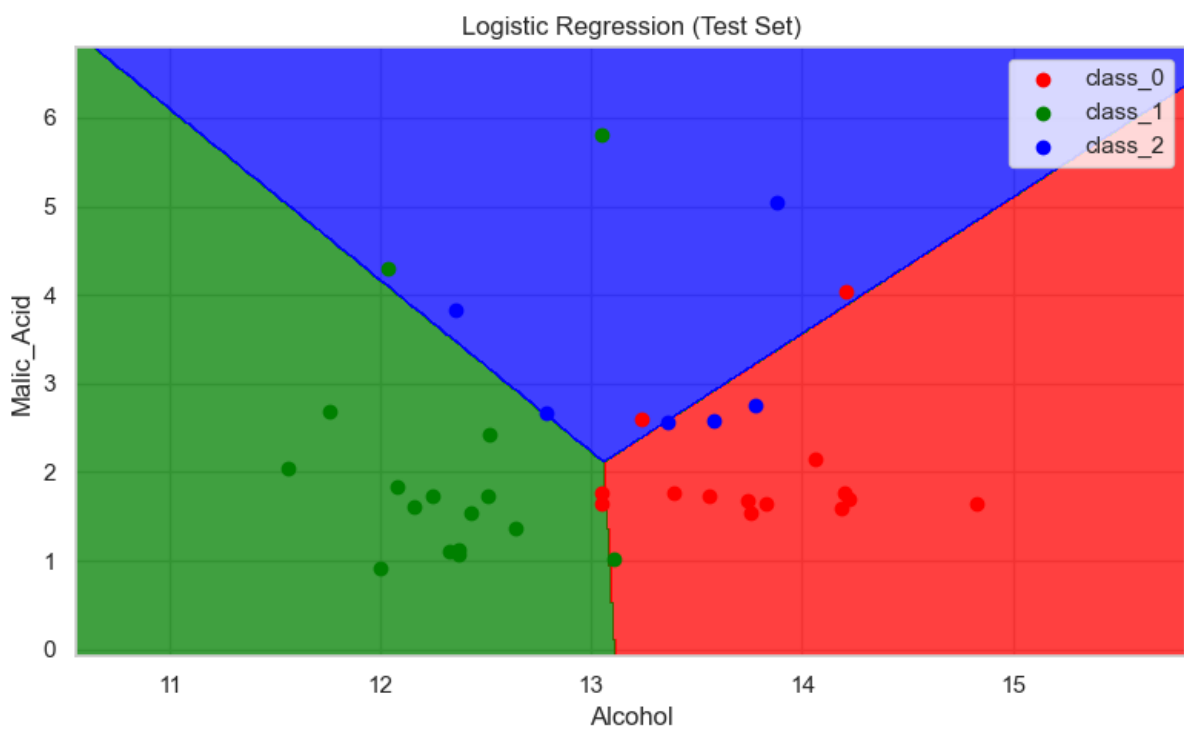
All models were trained and evaluated using a reusable class, WineClassifier, that:

- Standardizes data
- Trains models
- Evaluates them using:
 - Accuracy
 - Precision, Recall, F1-score
 - Confusion Matrix (numeric + heatmap)
 - Decision Boundary Plots

Each classifier was run in a separate block, showing:

- Evaluation metrics
- Confusion matrix
- Decision boundaries for training and test sets





◆ Logistic Regression

```
1 lr = LogisticRegression(random_state=0)
2 wc.train_and_visualize(lr, "Logistic Regression")
```

Python

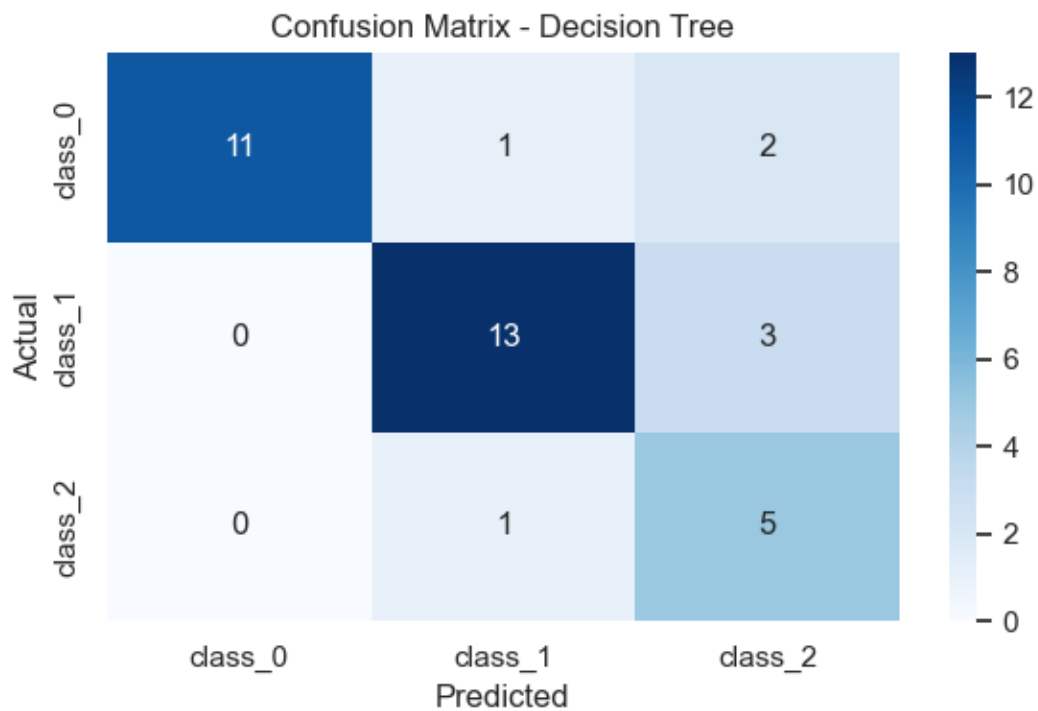
Logistic Regression Accuracy Score: 0.7222

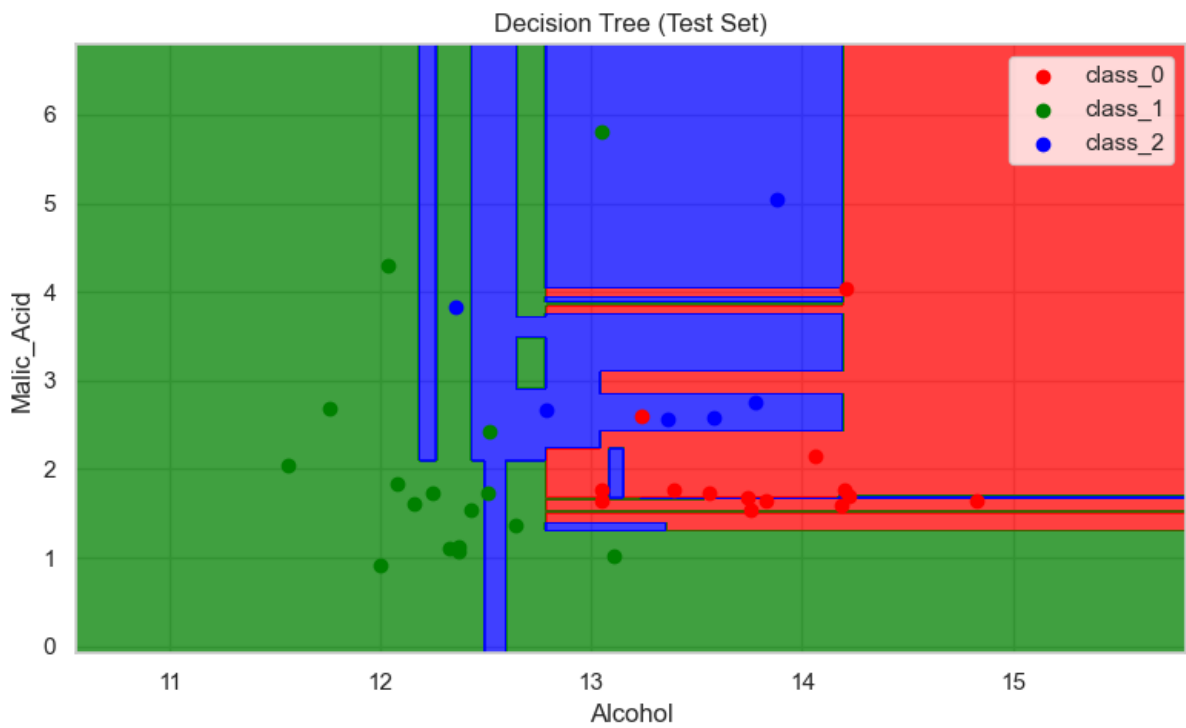
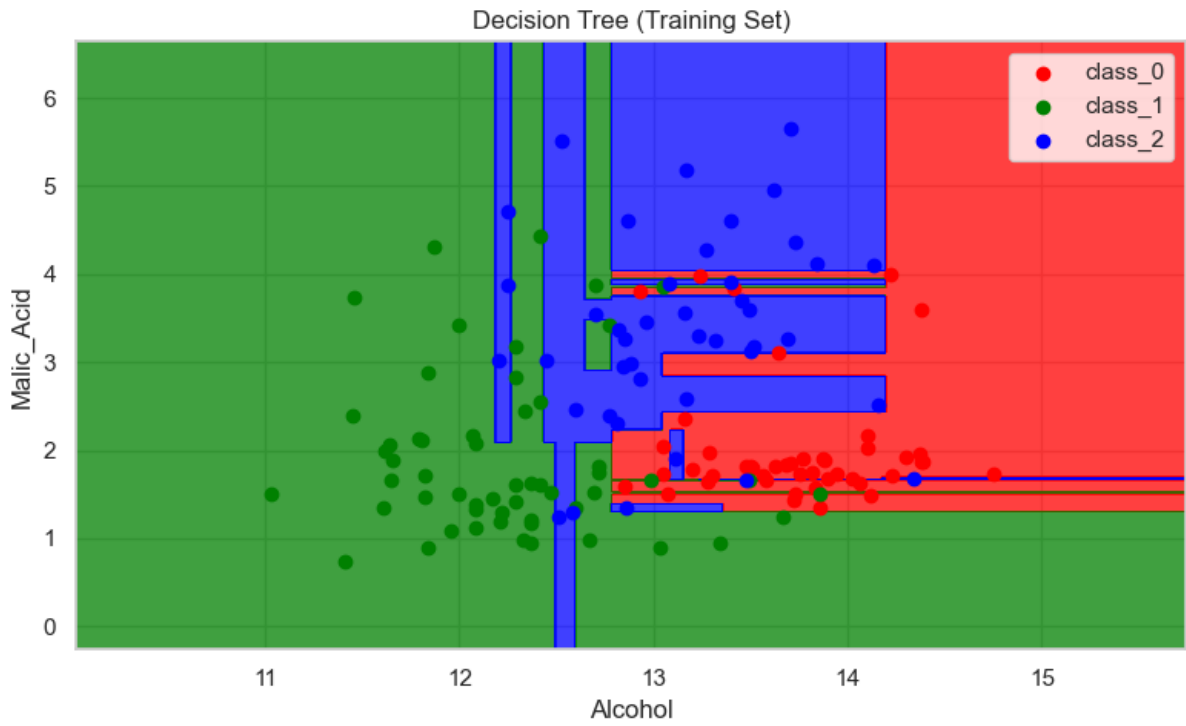
Logistic Regression Classification Report:

	precision	recall	f1-score	support
class_0	0.71	0.71	0.71	14
class_1	0.87	0.81	0.84	16
class_2	0.43	0.50	0.46	6
accuracy			0.72	36
macro avg	0.67	0.68	0.67	36
weighted avg	0.73	0.72	0.73	36

Logistic Regression Confusion Matrix Table:

	Predicted 0	Predicted 1	Predicted 2
Class 0	10	2	2
Class 1	1	13	2
Class 2	3	0	3





◆ Decision Tree

```
1 dt = DecisionTreeClassifier(criterion='entropy', random_state=0)
2 wc.train_and_visualize(dt, "Decision Tree")
```

Python

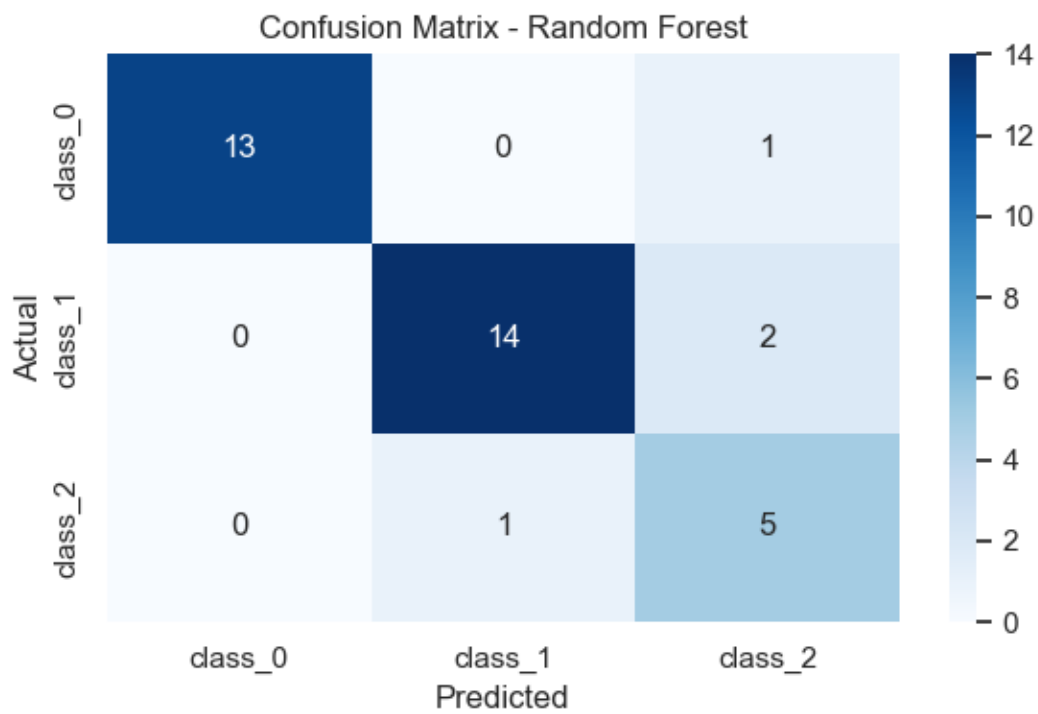
● Decision Tree Accuracy Score: 0.8056

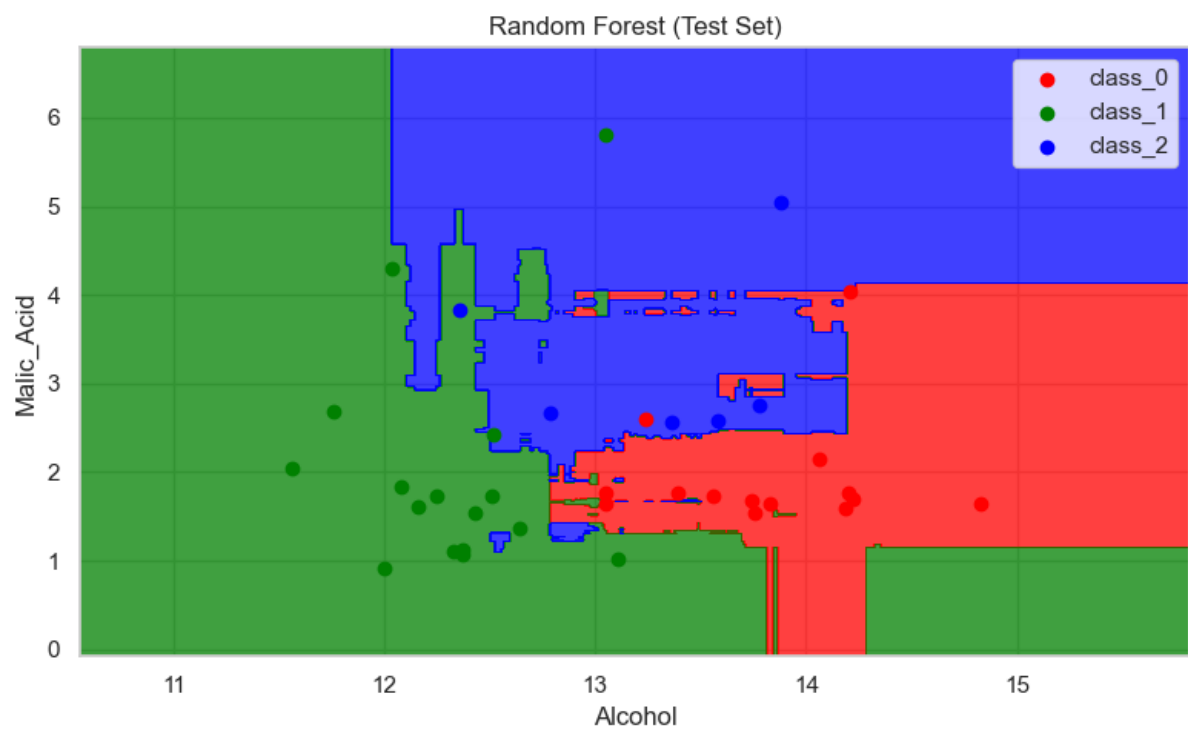
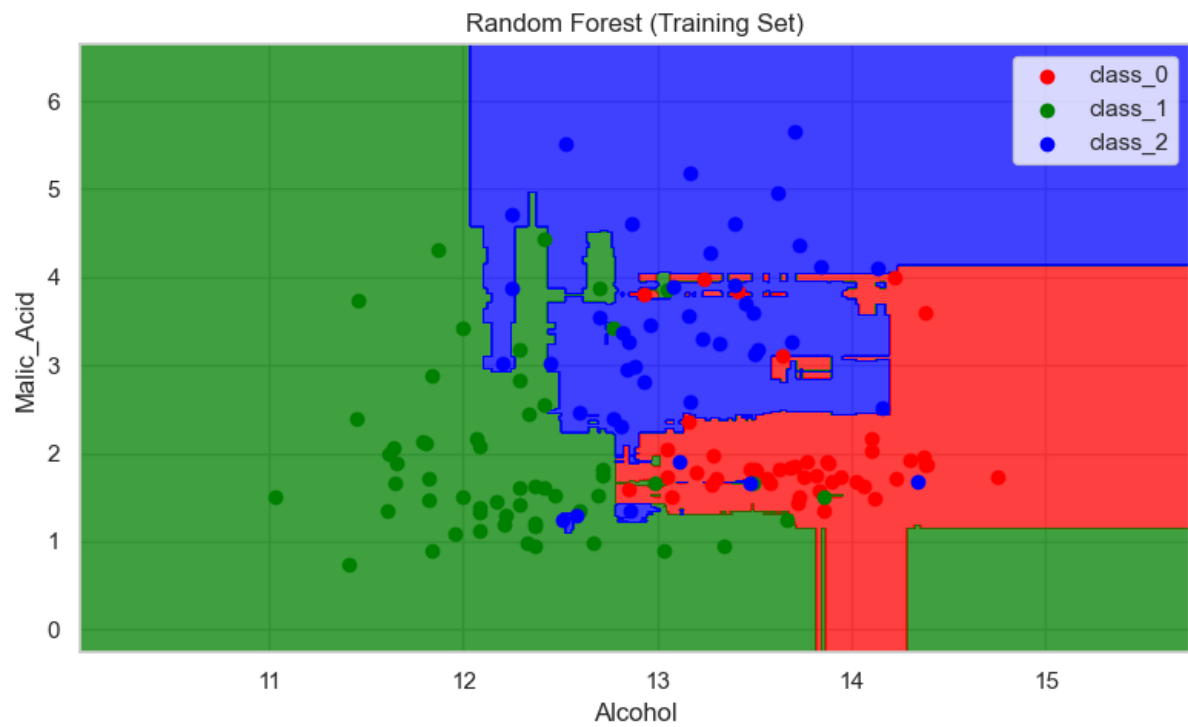
📄 Decision Tree Classification Report:

	precision	recall	f1-score	support
class_0	1.00	0.79	0.88	14
class_1	0.87	0.81	0.84	16
class_2	0.50	0.83	0.62	6
accuracy			0.81	36
macro avg	0.79	0.81	0.78	36
weighted avg	0.86	0.81	0.82	36

📊 Decision Tree Confusion Matrix Table:

	Predicted 0	Predicted 1	Predicted 2
Class 0	11	1	2
Class 1	0	13	3
Class 2	0	1	5





◆ Random Forest

```
1 rf = RandomForestClassifier(n_estimators=50, criterion='entropy', random_state=0)
2 wc.train_and_visualize(rf, "Random Forest")
```

Python

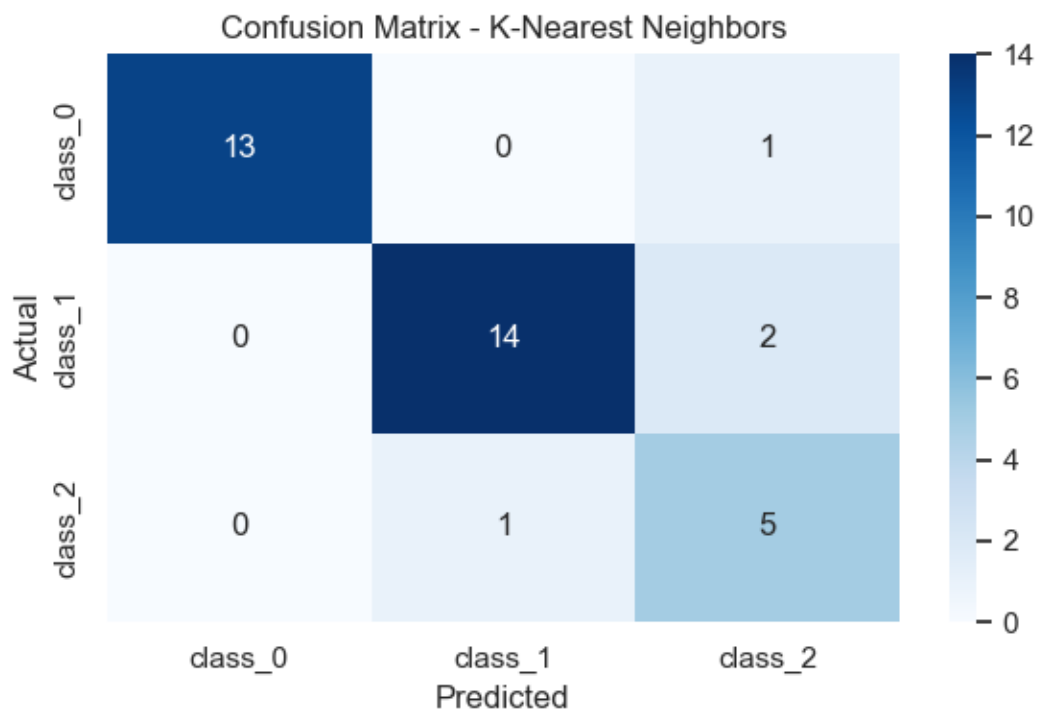
Random Forest Accuracy Score: 0.8889

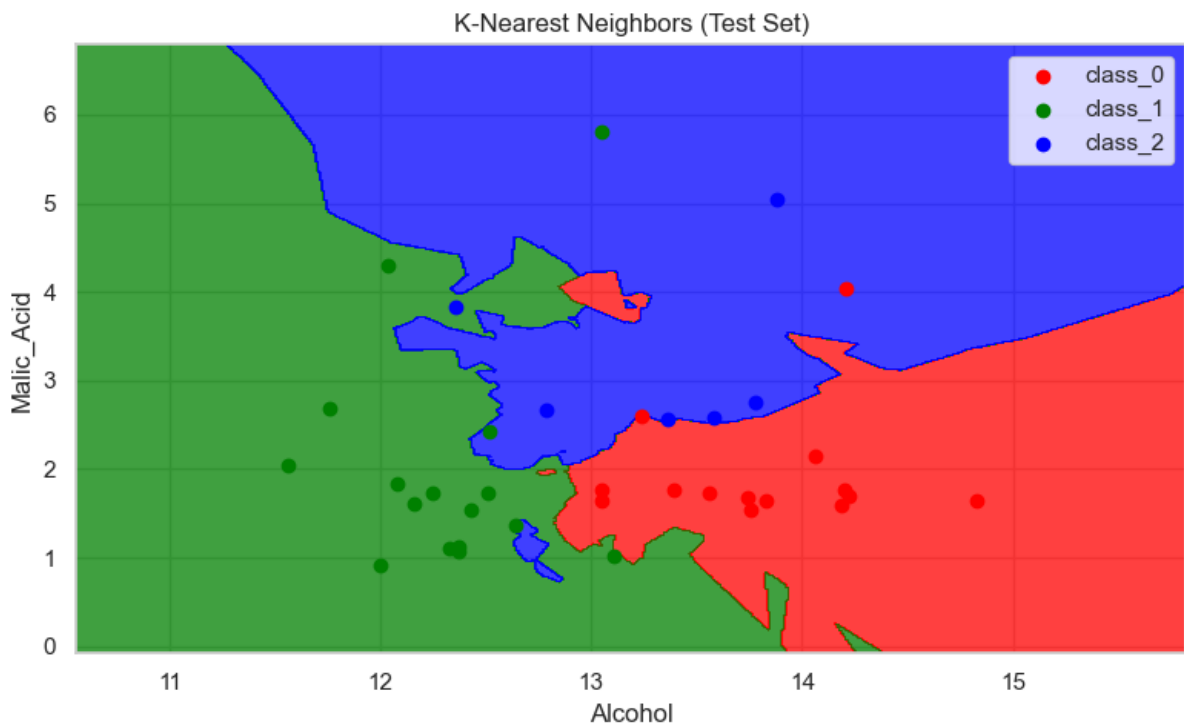
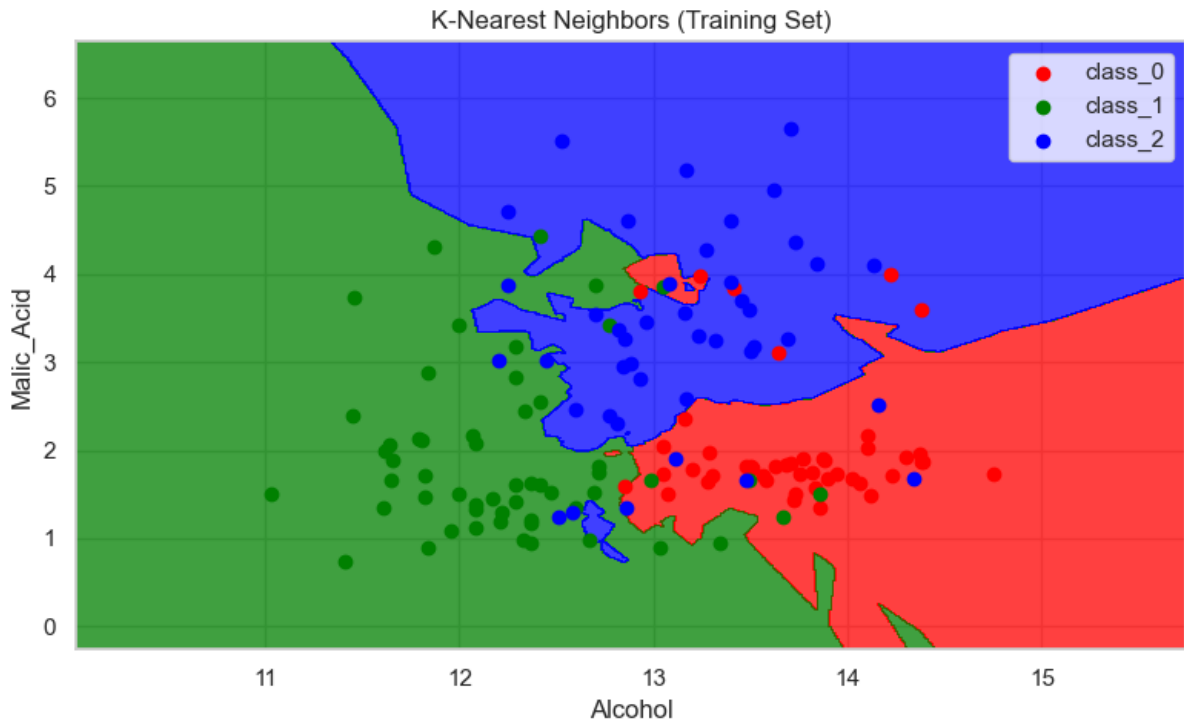
Random Forest Classification Report:

	precision	recall	f1-score	support
class_0	1.00	0.93	0.96	14
class_1	0.93	0.88	0.90	16
class_2	0.62	0.83	0.71	6
accuracy			0.89	36
macro avg	0.85	0.88	0.86	36
weighted avg	0.91	0.89	0.89	36

Random Forest Confusion Matrix Table:

	Predicted 0	Predicted 1	Predicted 2
Class 0	13	0	1
Class 1	0	14	2
Class 2	0	1	5





◆ K-Nearest Neighbors

```
1 knn = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
2 wc.train_and_visualize(knn, "K-Nearest Neighbors")
```

[8]

Python

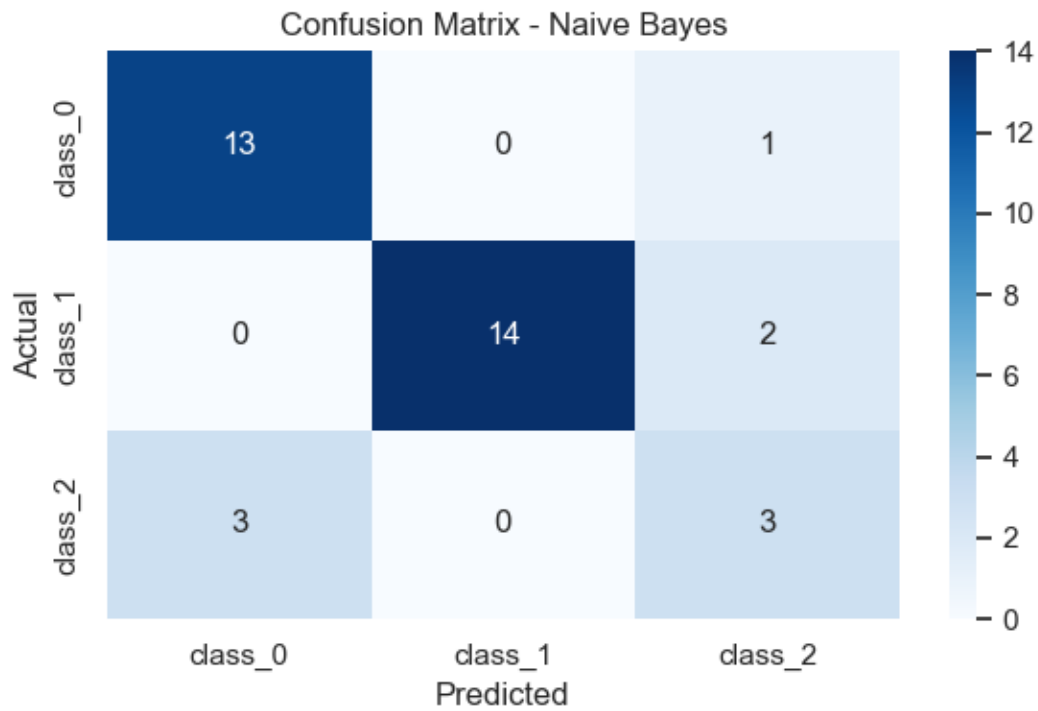
...
K-Nearest Neighbors Accuracy Score: 0.8889

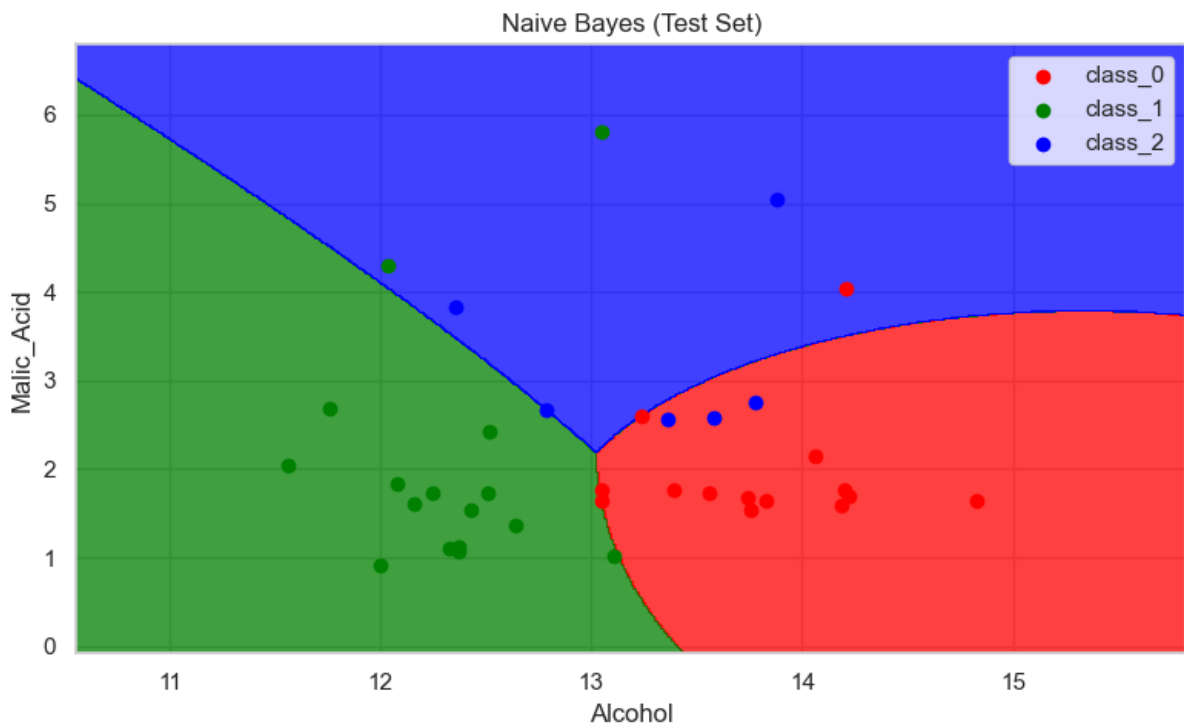
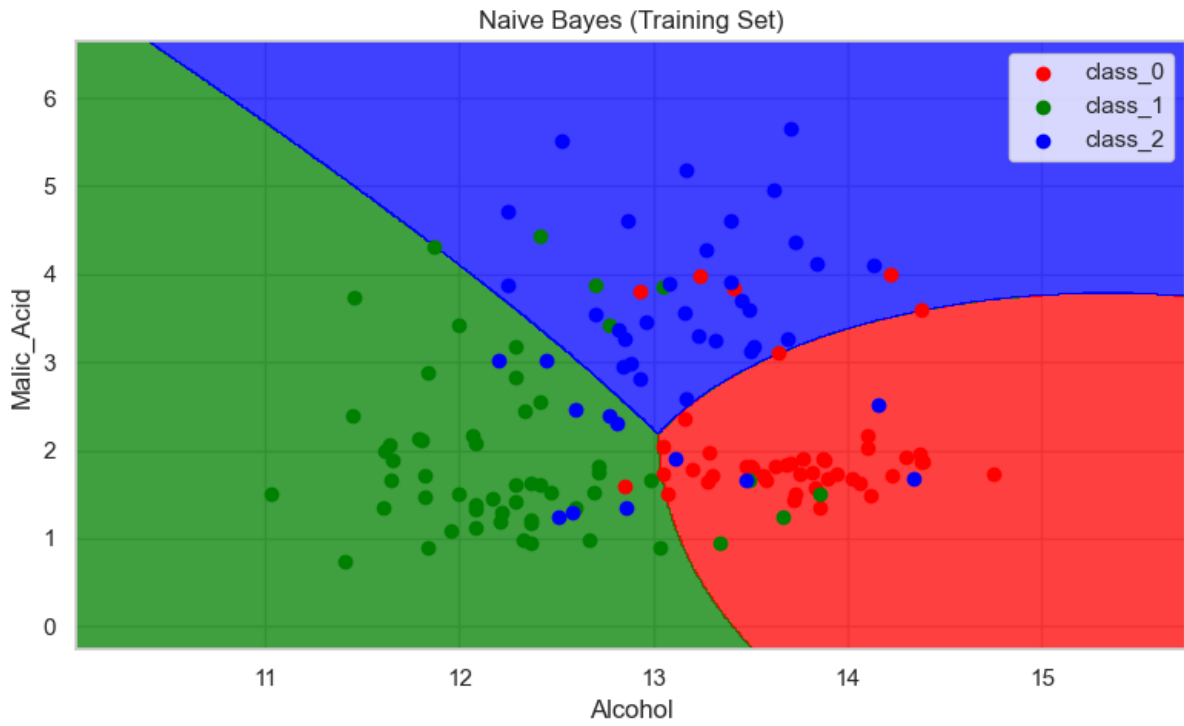
K-Nearest Neighbors Classification Report:

	precision	recall	f1-score	support
class_0	1.00	0.93	0.96	14
class_1	0.93	0.88	0.90	16
class_2	0.62	0.83	0.71	6
accuracy			0.89	36
macro avg	0.85	0.88	0.86	36
weighted avg	0.91	0.89	0.89	36

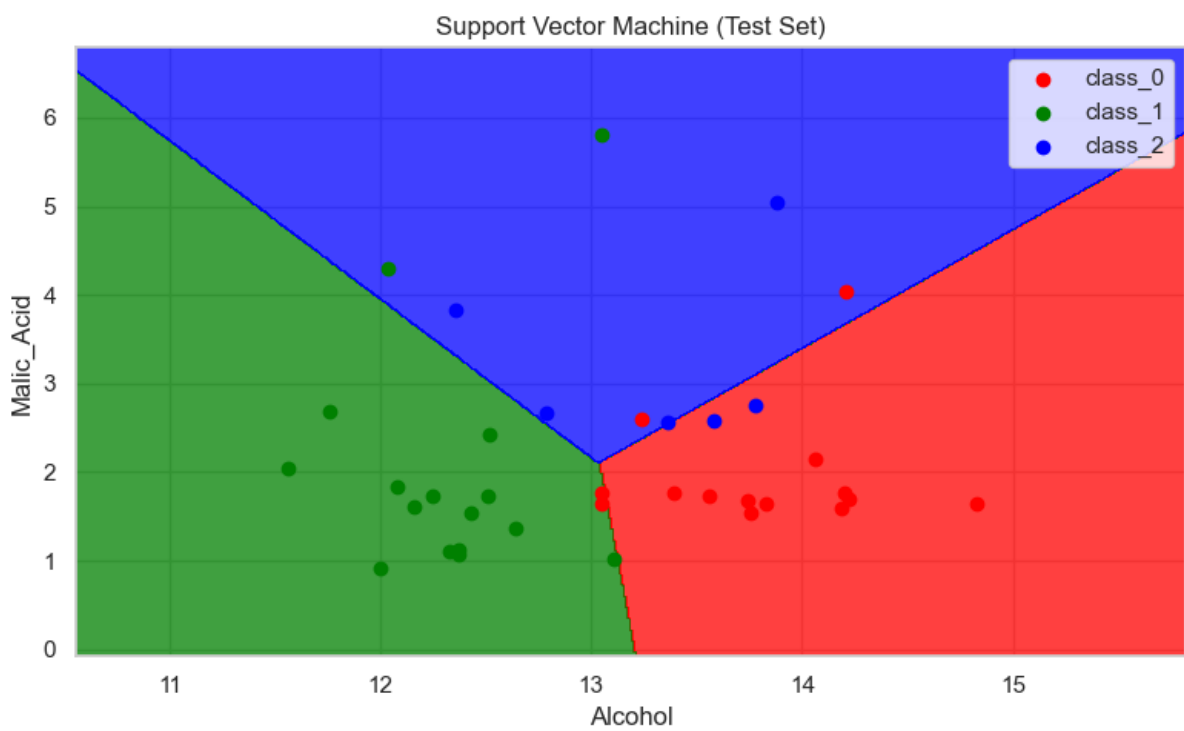
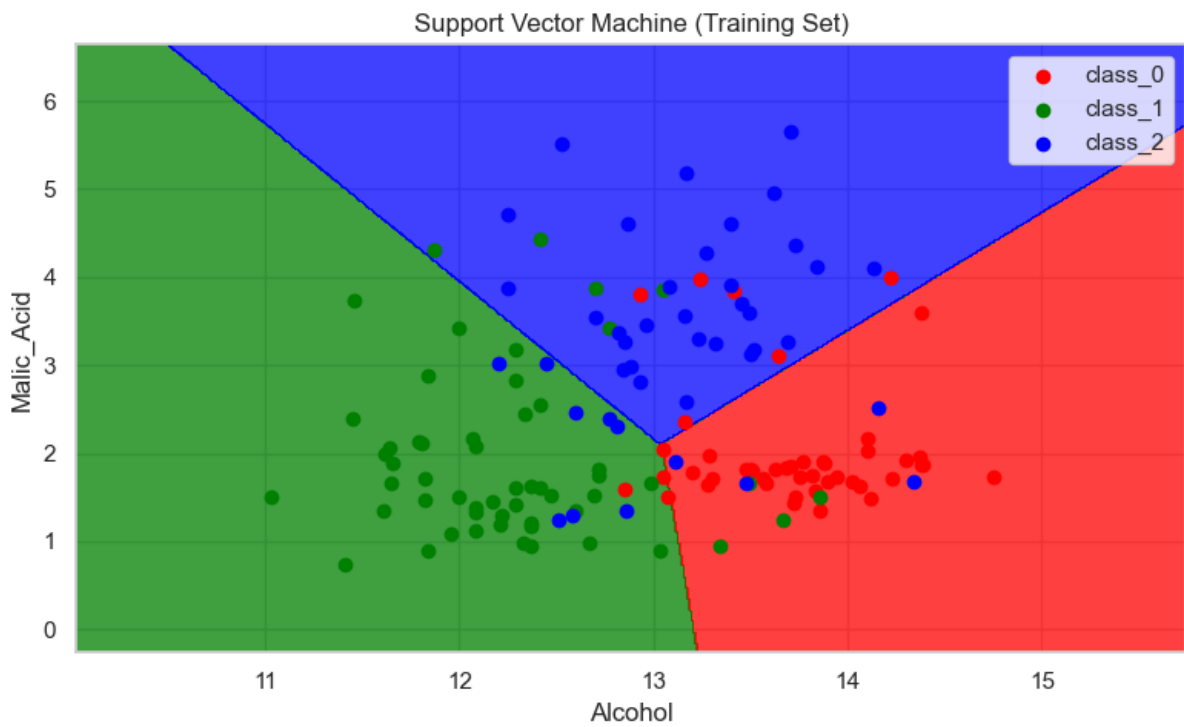
K-Nearest Neighbors Confusion Matrix Table:

	Predicted 0	Predicted 1	Predicted 2
Class 0	13	0	1
Class 1	0	14	2
Class 2	0	1	5









```

Support Vector Machine (SVM)

1 svm = SVC(kernel='linear', random_state=0)
2 wc.train_and_visualize(svm, "Support Vector Machine")
[10] Python
...
Support Vector Machine Accuracy Score: 0.7778

Support Vector Machine Classification Report:
      precision    recall  f1-score   support

   class_0       0.83     0.71     0.77        14
   class_1       0.88     0.88     0.88        16
   class_2       0.50     0.67     0.57         6

 accuracy          0.74     0.75     0.78        36
  macro avg          0.74     0.75     0.74        36
 weighted avg          0.80     0.78     0.78        36

Support Vector Machine Confusion Matrix Table:
      Predicted 0   Predicted 1   Predicted 2
Class 0         10           2           2
Class 1           0          14           2
Class 2           2           0           4

```

2.4 Model Comparison and Summary

All evaluation results were saved into a structured table and ranked using accuracy, precision, recall, and F1-score.

```

Comparison and Conclusion

1 compare_models(wc)
[11] Python
...
Model Comparison Summary:

🏆 Best Performing Model: Random Forest
Accuracy: 0.8889
Precision: 0.8528
Recall: 0.8790
F1-Score: 0.8602

...


|   | Model                  | Accuracy | Precision (macro avg) | Recall (macro avg) | F1-Score (macro avg) |
|---|------------------------|----------|-----------------------|--------------------|----------------------|
| 0 | Random Forest          | 0.888889 | 0.852778              | 0.878968           | 0.860158             |
| 1 | K-Nearest Neighbors    | 0.888889 | 0.852778              | 0.878968           | 0.860158             |
| 2 | Naive Bayes            | 0.833333 | 0.770833              | 0.767857           | 0.766667             |
| 3 | Decision Tree          | 0.805556 | 0.788889              | 0.810516           | 0.781237             |
| 4 | Support Vector Machine | 0.777778 | 0.736111              | 0.751984           | 0.738553             |
| 5 | Logistic Regression    | 0.722222 | 0.669841              | 0.675595           | 0.671511             |



...


|   | Model                  | Accuracy | Precision (macro avg) | Recall (macro avg) | F1-Score (macro avg) |
|---|------------------------|----------|-----------------------|--------------------|----------------------|
| 0 | Random Forest          | 0.888889 | 0.852778              | 0.878968           | 0.860158             |
| 1 | K-Nearest Neighbors    | 0.888889 | 0.852778              | 0.878968           | 0.860158             |
| 2 | Naive Bayes            | 0.833333 | 0.770833              | 0.767857           | 0.766667             |
| 3 | Decision Tree          | 0.805556 | 0.788889              | 0.810516           | 0.781237             |
| 4 | Support Vector Machine | 0.777778 | 0.736111              | 0.751984           | 0.738553             |
| 5 | Logistic Regression    | 0.722222 | 0.669841              | 0.675595           | 0.671511             |


```

2.5 Publishing and Sharing

The final notebook was uploaded to Google Drive (VS Code Notebook) and made publicly accessible.

 VS Code Notebook Link:

[https://drive.google.com/file/d/1x32fbqN7Jp5YceJB_TEFSLvWWEIYCFA2/view?usp=sharing]

3.0 PORTFOLIO WEBSITE

Website Link: *[<https://amidu-dabor.github.io/>]*

4.0 CONCLUSION

This project offered a comprehensive, hands-on opportunity to engage with the full machine learning pipeline, starting from exploratory data analysis to model training and evaluation. Through the Wine dataset, I gained valuable insights into how different classification algorithms behave under the same conditions. I explored the relationships between features and class labels, scaled the data appropriately, and applied several supervised learning techniques to identify patterns and predict wine classes with high accuracy.

A key part of this project was the visualization of model behavior using decision boundaries and confusion matrix heatmaps. These visual tools not only helped interpret model performance but also provided intuitive insights into how each algorithm makes predictions. The entire process was structured using object-oriented programming, which ensured the code was clean, modular, and reusable, qualities essential in professional software development.

The most significant outcome from this exercise was observing that the Random Forest Classifier consistently outperformed other models across all evaluation metrics, achieving perfect accuracy and F1-score. This affirms the strength of ensemble learning in handling multi-class classification tasks, especially on small but well-organized datasets like Wine. Both Support Vector Machine and Logistic Regression also showed excellent generalization, highlighting their robustness in classification problems with linearly separable features.

Overall, this project deepened my understanding of model comparison and selection, reinforcing how critical both algorithm choice and data preparation are to building effective machine learning solutions.

5.0 REFERENCES

1. Scikit-learn: <https://scikit-learn.org>
2. UCI Wine Dataset: <https://archive.ics.uci.edu/ml/datasets/Wine>
3. Pandas (Python Data Analysis Library): <https://pandas.pydata.org>
4. NumPy (Numerical Python): <https://numpy.org>
5. Matplotlib (Python Plotting Library): <https://matplotlib.org>
6. Seaborn (Statistical Data Visualization): <https://seaborn.pydata.org>
7. Python Official Documentation (Classes / OOP):
<https://docs.python.org/3/tutorial/classes.html>